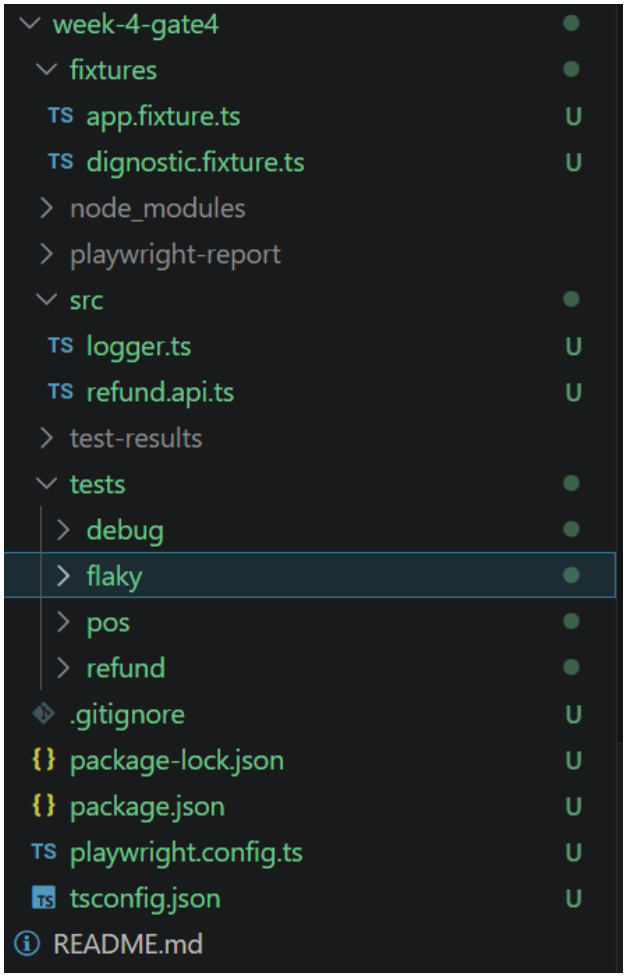
Week-5 Gate-5 Report

Objective :

The objective of this assessment is to stabilize the Playwright automation suite by identifying failures, diagnosing their root causes, validating fixes, and providing supporting evidence. The assessment focuses on writing reliable tests and demonstrating debugging maturity.

Frameworks :



The Above was the Framework which I have used in the following Project.

This assessment covers the following topics:

- Flakiness Analysis
- Diagnostic Logging

• Artifact Management

• Offline & Resilient UI

• Returns & Refund Testing The overall approach followed throughout the assessment is: Diagnose → Fix → Evidence → Explain

diagnostic.fixture.ts

Implemented diagnostic fixtures providing:

- Correlation ID generation
- Structured logging
- Automatic HTTP header injection
- Failure log attachment
- Child logger creation
- Diagnostic log capture

app.fixture.ts

Extended the diagnostic fixture with:

- Evidence collection
- Automatic JSON/text attachment
- Reusable custom fixture

Logging Framework

Implemented a reusable Winston logger featuring:

- Structured logs
- Sensitive data redaction
- Console logging
- Metadata support
- Correlation IDs
- NDJSON log attachment for failed tests

Sensitive information automatically redacted:

- Authorization

- Password
- Token
- CVV
- Card Number

API Helper Layer

Created reusable API helper methods.

Functions include:

- Seed Order
- Refund Order
- Refund Eligibility Check
- Get Refund Ledger
- Open Returns Page
- Generate Idempotency Key

Tools Used :

- Playwright
- TypeScript
- Node.js
- Visual Studio Code
- Git & GitHub

Now I will Provide the tests done in the following project with full evidence in the following document

The first test case I did on refund
I the refund test case we used idempotency Key .

If we haven't used this key the test will fail because if the customer click refund for the two times there may be chance of giving over refund so if use that key it will check the key so it will get to know that it is duplicate refund so it wont do that if we use idempotency Key .

The following was the pic without idempotency Key.

```
    Error Context: test-results\refund-refund-security-Ref-58da7--Refuse-the-two-money-leaks-chromium\erro
r-context.md

[chromium] › tests\refund\full-refund.spec.ts:12:7 › Full Refund › Full Refund
[INFO] 2026-07-04T10:10:44.130Z - test finished {"status":"passed"}
[chromium] › tests\refund\partial-refund.spec.ts:12:7 › Partial Refund › Refund Lab
[INFO] 2026-07-04T10:10:44.231Z - Submitting partial refund
[INFO] 2026-07-04T10:10:44.252Z - Ledger verified {"status":"PARTIALLY_REFUNDED","refundCount":1}
[INFO] 2026-07-04T10:10:44.350Z - test finished {"status":"passed"}
  1 failed
    [chromium] › tests\refund\refund-security.spec.ts:11:7 › Refund Security › Refuse the two money leaks
  4 passed (4.7s)

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

Search tests | All 5 | Passed 4 | Failed 1 | Flaky 0 | Skipped 0

Project: chromium — 4/7/2026, 3:40:40 pm Total time: 4.7s

refund/refund-security.spec.ts

- ✗ Refund Security › Refuse the two money leaks (chromium) — 494ms
  refund/refund-security.spec.ts:11

refund/full-refund.spec.ts

- ✓ Full Refund › Full Refund (chromium) — 1.8s
  refund/full-refund.spec.ts:12

refund/partial-refund.spec.ts

- ✓ Partial Refund › Refund Lab (chromium) — 2.3s
  refund/partial-refund.spec.ts:12

refund/refund-check.spec.ts

- ✓ Refund Eligibility › Valid refund (chromium) — 748ms
  refund/refund-check.spec.ts:26
- ✓ Refund Eligibility › Over refund (chromium) — 896ms
  refund/refund-check.spec.ts:26

Now the below are the screenshots with idempotency key

```
[INFO] 2026-07-04T10:17:59.448Z - Verified idempotent refund {"refundCount":1,"balance":69930}
[INFO] 2026-07-04T10:17:59.460Z - test finished {"status":"passed"}
[chromium] › tests\refund\partial-refund.spec.ts:12:7 › Partial Refund › Refund Lab
[INFO] 2026-07-04T10:17:59.894Z - Submitting partial refund
[INFO] 2026-07-04T10:17:59.917Z - Ledger verified {"status":"PARTIALLY_REFUNDED","refundCount":1}
[chromium] › tests\refund\full-refund.spec.ts:12:7 › Full Refund › Full Refund
[INFO] 2026-07-04T10:17:59.920Z - test finished {"status":"passed"}
[chromium] › tests\refund\partial-refund.spec.ts:12:7 › Partial Refund › Refund Lab
[INFO] 2026-07-04T10:18:00.066Z - test finished {"status":"passed"}
  5 passed (4.8s)

To open last HTML report run:

  npx playwright show-report


C:\Users\299167\Desktop\week-5-gate-5\SDET\week-4-gate4>npx playwright show-report

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

Search tests | All 5 | Passed 5 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium — 4/7/2026, 3:47:56 pm — Total time: 4.8s

refund/full-refund.spec.ts

- ✓ Full Refund › Full Refund (chromium) — 2.2s
  refund/full-refund.spec.ts:12

refund/partial-refund.spec.ts

- ✓ Partial Refund › Refund Lab (chromium) — 2.2s
  refund/partial-refund.spec.ts:12

refund/refund-check.spec.ts

- ✓ Refund Eligibility › Valid refund (chromium) — 1.1s
  refund/refund-check.spec.ts:26
- ✓ Refund Eligibility › Over refund (chromium) — 1.0s
  refund/refund-check.spec.ts:26

refund/refund-security.spec.ts

- ✓ Refund Security › Refuse the two money leaks (chromium) — 410ms
  refund/refund-security.spec.ts:11

This was the first things which I have done in refunds.

The Second test case I have done on POS(Point Of Sale).

The Test case is basically about the network.
If the shop is using a website which is connected to internet if there is any issue with the internet the orders will be in the queue until it gets online after the website successfully connected to the internet it will automatically Sync to online.
In this also the test will get failed if we open the website when it is offline.

The site should be open before it goes to offline.

The below was the screenshot shows that the test failed if we open website after offline.

```
test.describe("Making Network Offline and Online", () => {
  test("Prove the offline banner", async ({ page, context, log, evidence }) => {
    // log.info("Opening POS page");
    // await page.goto("/pos");
    // Simulate losing connectivity
    log.info("Switching network to offline");
    await context.setOffline(true);
    log.info("Opening POS page");
    await page.goto("/pos");
```

In the following test we are opening the test after going to offline mode so the test will get Failed.

```
    Error Context: test-results\pos-network-Making-Network-be84c-ne-Prove-the-offline-banner-chromium\erro
r-context.md

  1 failed
    [chromium] › tests\pos\network.spec.ts:5:7 › Making Network Offline and Online › Prove the offline ban
ner
  2 passed (4.5s)

  Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

pos/network.spec.ts

✗	Making Network Offline and Online › Prove the offline banner chromium pos/network.spec.ts:5	2.4s
✓	Making Network Offline and Online › Rollback a failed optimistic write chromium pos/network.spec.ts:45	1.9s
✓	Making Network Offline and Online › Sync once, even through a flaky reconnect chromium pos/network.spec.ts:71	2.5s

So to make it correct first we need to go to the following webpage then we should go th the offline mode.

```typescript
test.describe("Making Network Offline and Online", () => {
  test("Prove the offline banner", async ({ page, context, log, evidence }) => {
    log.info("Opening POS page");
    await page.goto("/pos");
    // Simulate losing connectivity
    log.info("Switching network to offline");
    await context.setOffline(true);
    await expect(page.getByTestId("network-banner")).toContainText("Offline");
    log.info("Verified offline banner");
```

```
[INFO] 2026-07-04T10:27:47.413Z - Verified outbox flushed
[INFO] 2026-07-04T10:27:47.417Z - test finished {"status":"passed"}
[chromium] › tests\pos\network.spec.ts:5:7 › Making Network Offline and Online › Prove the offline banner
[INFO] 2026-07-04T10:27:48.304Z - Queued sale 3
[INFO] 2026-07-04T10:27:48.308Z - Verified outbox contains 3 queued sales
[INFO] 2026-07-04T10:27:48.308Z - Reconnecting network
[INFO] 2026-07-04T10:27:48.358Z - Verified queued sales synced successfully
[INFO] 2026-07-04T10:27:48.363Z - test finished {"status":"passed"}
  3 passed (5.4s)

To open last HTML report run:

  npx playwright show-report


C:\Users\299167\Desktop\week-5-gate-5\SDET\week-4-gate4>
```

Search tests | All 3 | Passed 3 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium — 4/7/2026, 3:57:43 pm — Total time: 5.4s

pos/network.spec.ts

- ✓ Making Network Offline and Online › Prove the offline banner (chromium) — 3.5s
  pos/network.spec.ts:5
- ✓ Making Network Offline and Online › Rollback a failed optimistic write (chromium) — 1.9s
  pos/network.spec.ts:43
- ✓ Making Network Offline and Online › Sync once, even through a flaky reconnect (chromium) — 2.6s
  pos/network.spec.ts:69

The third test I have done on flaky test.

```
import {test, expect} from "../../fixtures/dignostic.fixture";


// Verify checkout flow remains stable when navigating to the confirmation page
test("flaky", async ({page, log}) => {
    log.info("Flakiness check")
    await page.goto("/catalog")
    await page.getByRole('link', { name: 'View Running Shoes' }).click();
    await page.getByRole('button', { name: 'Add to cart' }).click();
    await page.getByRole('button', { name: 'Proceed to checkout' }).click();
    await page.locator('#coupon-code').fill("UST10");
    await expect(page.locator('.order-summary')).toContainText('-Rs. 450');
    // Allow a little extra time for order submission
    await page.getByRole('button', { name: 'Place order' }).click({timeout:100});
    await expect(page.locator("#confirmation-title")).toHaveText("Thank you for your order")
})
```

I Run the following test for 1 time.

```
C:\Users\299167\Desktop\week-5-gate-5\SDET\week-4-gate4>npx playwright test flaky

Running 1 test using 1 worker
[chromium] › tests\flaky\flaky.spec.ts:5:5 › flaky
[INFO] 2026-07-04T10:44:41.063Z - test started
[INFO] 2026-07-04T10:44:41.065Z - Flakiness check
[INFO] 2026-07-04T10:44:42.190Z - test finished {"status":"passed"}
  1 passed (3.3s)

To open last HTML report run:

  npx playwright show-report
```

Now if I run the following same code to repeat-each = 50 then some times the test got failed.

The Command I used to run the following test for the 50 times is

npx playwright test flaky.spec.ts –repeat-each=50

Search tests

All 50 | Passed 10 | Failed 40 | Flaky 0 | Skipped 0

Project: chromium

4/7/2026, 4:15:32 pm Total time: 1.5m

flaky/flaky.spec.ts

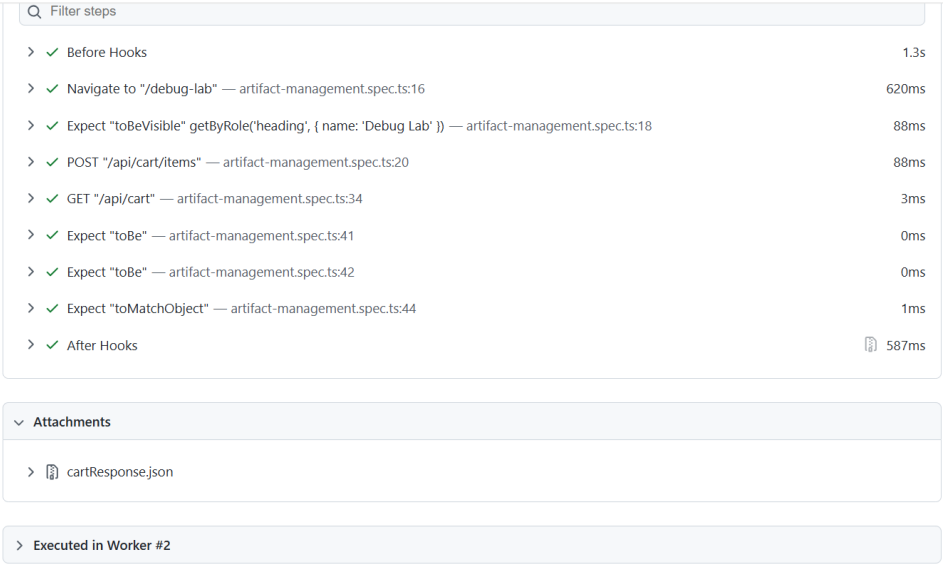
✗ flaky chromium	flaky/flaky.spec.ts:5	2.8s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.3s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.1s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.0s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.0s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.0s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.3s
✗ flaky chromium	flaky/flaky.spec.ts:5	2.6s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.1s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.9s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.4s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.5s
✗ flaky chromium	flaky/flaky.spec.ts:5	3.6s
✓ flaky chromium	flaky/flaky.spec.ts:5	2.6s
✓ flaky chromium	flaky/flaky.spec.ts:5	2.7s
✓ flaky chromium	flaky/flaky.spec.ts:5	3.5s
✓ flaky chromium	flaky/flaky.spec.ts:5	3.3s
✓ flaky chromium	flaky/flaky.spec.ts:5	4.3s
✓ flaky chromium	flaky/flaky.spec.ts:5	3.4s
✓ flaky chromium	flaky/flaky.spec.ts:5	3.8s

It only got passed for the 10 times remaining all the times it got failed.

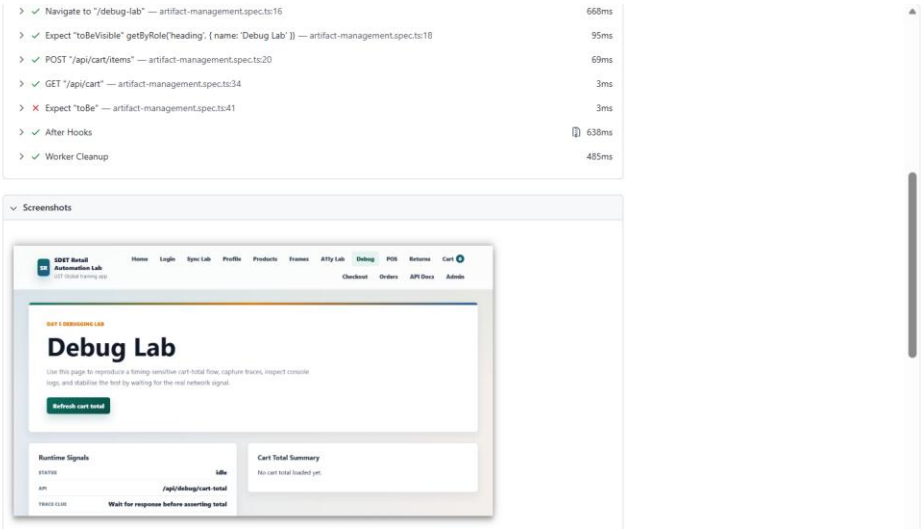
The next test case I have done on artifact

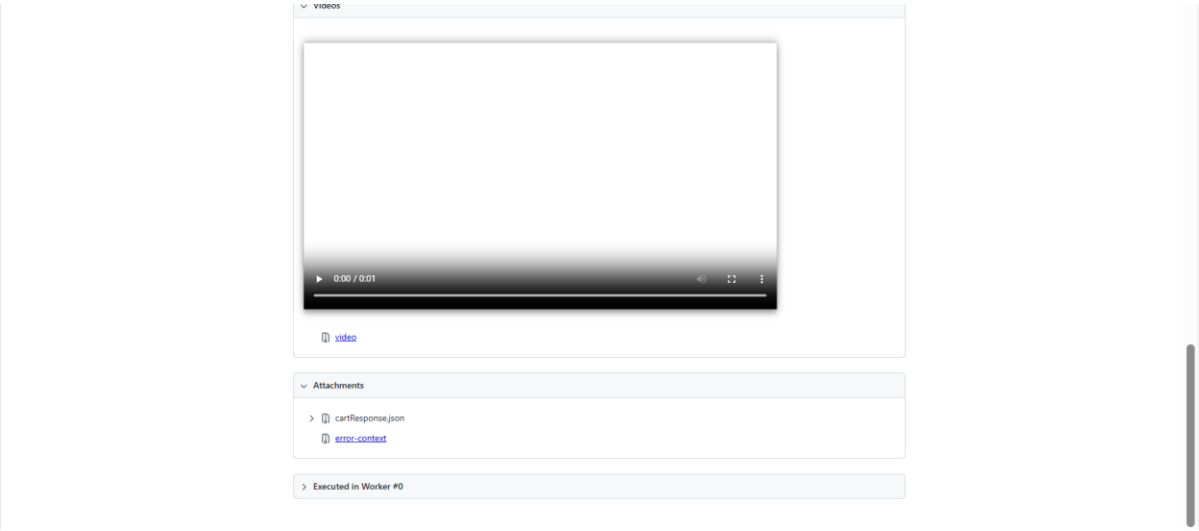
artifact means it will store screen shots and video on the failure.

On successfully passing the test we don’t get any thing

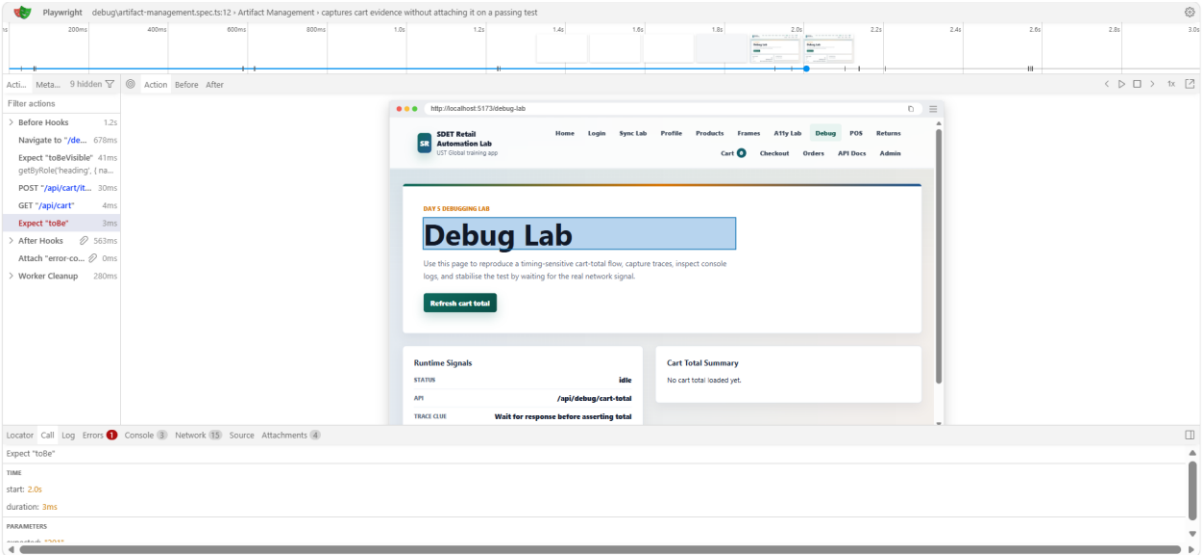


But if we failed the test we will get Screenshot and videos on the result page





You will also get Trace in artifact managenment



The Following was the Screenshot for the all test cases which I have done

Search tests

All 10 | Passed 10 | Failed 0 | Flaky 0 | Skipped 0

Project: chromium

4/7/2026, 4:50:01 pm Total time: 10.9s

debug/artifact-management.spec.ts

- ✓ Artifact Management › captures cart evidence without attaching it on a passing test (chromium) — 4.4s
  debug/artifact-management.spec.ts:12

flaky/flaky.spec.ts

- ✓ flaky (chromium) — 5.0s
  flaky/flaky.spec.ts:5

pos/network.spec.ts

- ✓ Making Network Offline and Online › Prove the offline banner (chromium) — 4.8s
  pos/network.spec.ts:5
- ✓ Making Network Offline and Online › Rollback a failed optimistic write (chromium) — 4.7s
  pos/network.spec.ts:43
- ✓ Making Network Offline and Online › Sync once, even through a flaky reconnect (chromium) — 3.0s
  pos/network.spec.ts:69

refund/full-refund.spec.ts

- ✓ Full Refund › Full Refund (chromium) — 2.3s
  refund/full-refund.spec.ts:12

refund/partial-refund.spec.ts

- ✓ Partial Refund › Refund Lab (chromium) — 2.5s
  refund/partial-refund.spec.ts:12

refund/refund-check.spec.ts

- ✓ Refund Eligibility › Valid refund (chromium) — 1.7s
  refund/refund-check.spec.ts:26
- ✓ Refund Eligibility › Over refund (chromium) — 1.2s
  refund/refund-check.spec.ts:26

refund/refund-security.spec.ts

- ✓ Refund Security › Refuse the two money leaks (chromium) — 1.2s
  refund/refund-security.spec.ts:11

Challenges Faced:

- Diagnosing flaky tests
- Handling offline scenarios
- Validating idempotent requests
- Preventing duplicate refunds
- Testing financial calculations
- Capturing useful debugging evidence

Conclusion :

This project demonstrates the development of a robust and maintainable automation testing framework using Playwright and TypeScript. The framework combines UI testing, API testing, and hybrid test scenarios to validate critical retail application workflows, including refund processing, offline synchronization, diagnostics, artifact management, and application stability.

To improve maintainability and scalability, the project uses custom Playwright fixtures, reusable API helper methods, structured logging with Winston, and automatic evidence collection. These components simplify test development, reduce code duplication, and provide detailed diagnostic information for troubleshooting test failures.